



Containerizing Models for Deployment

Package your ML models with containers for consistent runtime everywhere - from cloud to edge deployments

Why Containerize Models?

Containerization eliminates the infamous "**works on my machine**" problem that plagues ML deployments while providing:

Portability

Run the same container across any cloud provider or local development environment

Reproducibility

Ensure consistent training and inference environments across teams

MLOps Integration

Essential building block for automated ML pipelines and deployments



Containerization Workflow

Prepare Model Artifacts

Collect model weights, configuration files, and tokenizers in a deployment-ready format

Write Inference Script

Develop API endpoints using FastAPI, Flask, or framework-specific servers like TorchServe

Create Dockerfile

Define runtime environment with all dependencies and configuration

Build & Tag Image

Create versioned container images that package your model and code

Push to Registry

Store images in DockerHub, AWS ECR, GCP Artifact Registry, or Azure ACR

Deploy

Run containers on Kubernetes, serverless platforms, or edge devices

Anatomy of a Dockerfile (AI Model)

```
FROM pytorch/pytorch:2.1.0-cuda11.8-cudnn8-runtime
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY model/ ./model
COPY app.py .
CMD ["uvicorn", "app:app", "--host", "0.0.0.0",
    "--port", "8080"]
```



Base Image

Start with PyTorch + CUDA for GPU support



Dependencies

Install Python packages from requirements.txt



Copy Assets

Add model artifacts and inference code



Entry Point

Run web server to handle inference requests

Each instruction creates a new layer in the container image



Model Packaging Options



TorchScript

Serialized PyTorch models for production deployment

- Optimized for PyTorch runtime
- Supports both eager and graph execution



ONNX

Open Neural Network Exchange format for cross-framework compatibility

- Framework-agnostic representation
- Wide tooling ecosystem support



TensorRT

NVIDIA's high-performance inference optimizer

- Kernel fusion and precision calibration
- Best for NVIDIA hardware



SavedModel (TF)

TensorFlow's native serialization format

- Full computation graph + variables
- Works with TF Serving

Choose the format that best balances performance and compatibility requirements for your deployment



Inference Server Choices

Select the right inference server based on your framework, performance needs, and operational requirements:

FastAPI / Flask

Lightweight Python web frameworks

- Simple to implement and customize
- Great for prototyping and small-scale deployments
- Full control over request handling

NVIDIA Triton

Multi-framework, GPU-optimized server

- Supports TensorFlow, PyTorch, ONNX, TensorRT
- Dynamic batching for throughput optimization
- Multi-GPU and multi-node scaling

TorchServe

Production-ready PyTorch serving

- Model versioning and A/B testing
- Metrics and logging built-in
- Optimized for PyTorch models

TF Serving

TensorFlow's native serving system

- Optimized for TensorFlow models
- Versioning and rollback capabilities
- High-performance C++ implementation

Registry & Versioning

Container Registries

Public

DockerHub

- Free for public images
- Simple sharing across teams

Private Cloud

AWS ECR, GCP Artifact Registry, Azure ACR

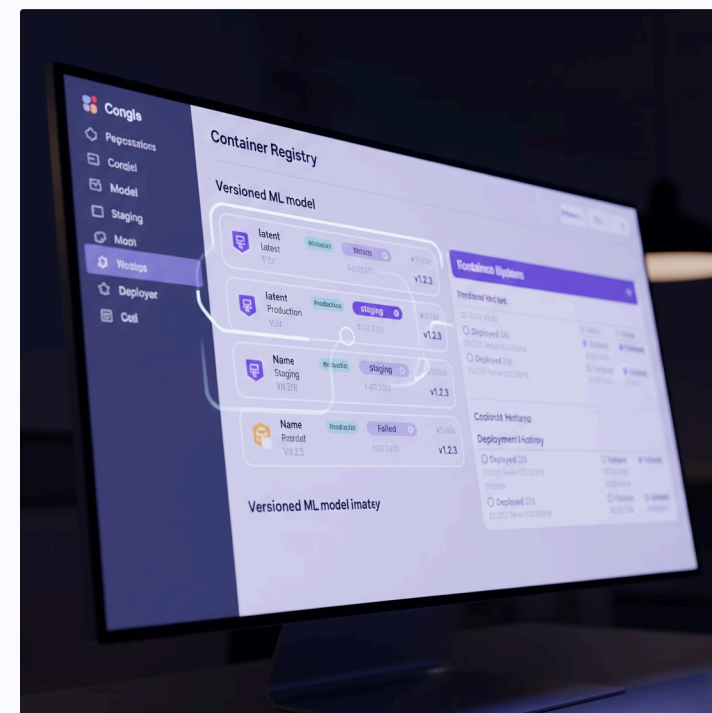
- Integrated with cloud IAM
- Regional deployments

Image Tagging Strategies

`my-model:latest` - Always points to newest version

`my-model:v1.0`, `my-model:v1.1` - Specific versions

`my-model:20231025-gpu` - Date + feature tags



Proper versioning enables:

- Quick rollbacks when deployments fail
- A/B testing different model versions
- Traceability from training to production



Multi-Stage Builds (Optimization)

Benefits

- Dramatically reduced image size
- Faster deployments and scaling
- Improved security (fewer vulnerabilities)
- Lower storage and bandwidth costs

```
FROM python:3.9 AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --user -r requirements.txt
RUN python -m pip install onnxruntime
```

```
FROM python:3.9-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY ./model /app/model
COPY ./app.py /app/
ENV PATH=/root/.local/bin:$PATH
CMD ["uvicorn", "app:app", "--host", "0.0.0.0"]
```

The [builder stage](#) handles all dependencies, while the [runtime stage](#) only includes what's needed for inference

GPU-Enabled Containers

Requirements for GPU Access



NVIDIA Container Toolkit

Install on host to enable GPU passthrough to containers



Compatible Base Image

Use image with matching CUDA + cuDNN versions



Driver Compatibility

Container CUDA $\leq$ host driver capability

Testing GPU Availability

```
docker run --gpus all \
nvidia/cuda:11.8.0-base-ubuntu22.04 \
nvidia-smi
```



Multi-GPU Deployments

For scaling across multiple GPUs:

- Kubernetes with [device plugins](#)
- Resource allocation via [node selectors](#)
- Fractional GPU allocation with [MIG](#) (A100/H100)

Case Study: NLP Model Deployment

Challenge

Deploy fine-tuned DistilBERT for high-throughput sentiment analysis with strict latency requirements

Solution

01

Model Optimization

Exported PyTorch model to ONNX format for cross-platform performance

02

API Development

FastAPI endpoints with async request handling and batching support

03

Containerization

Multi-stage Docker build reducing image size from 4.2GB to 1.8GB

04

Deployment Architecture

GKE cluster with horizontal pod autoscaling based on CPU/memory metrics

Results

<300ms

Latency

P95 response time under load

120

RPS

Requests per second per pod

99.9%

Availability

Service uptime

Case Study: Vision Model Edge Deployment



Challenge

Deploy YOLOv8 model on resource-constrained edge devices for real-time object detection

Solution

Model Optimization

INT8 quantization + TensorRT export reduced model size by 75%

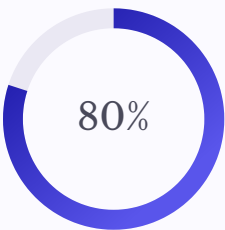
Edge Container

Built for NVIDIA Jetson platform with L4T base image

Deployment

Containerized runtime deployed via edge management platform

Results



Latency Reduction

From 200ms → 40ms

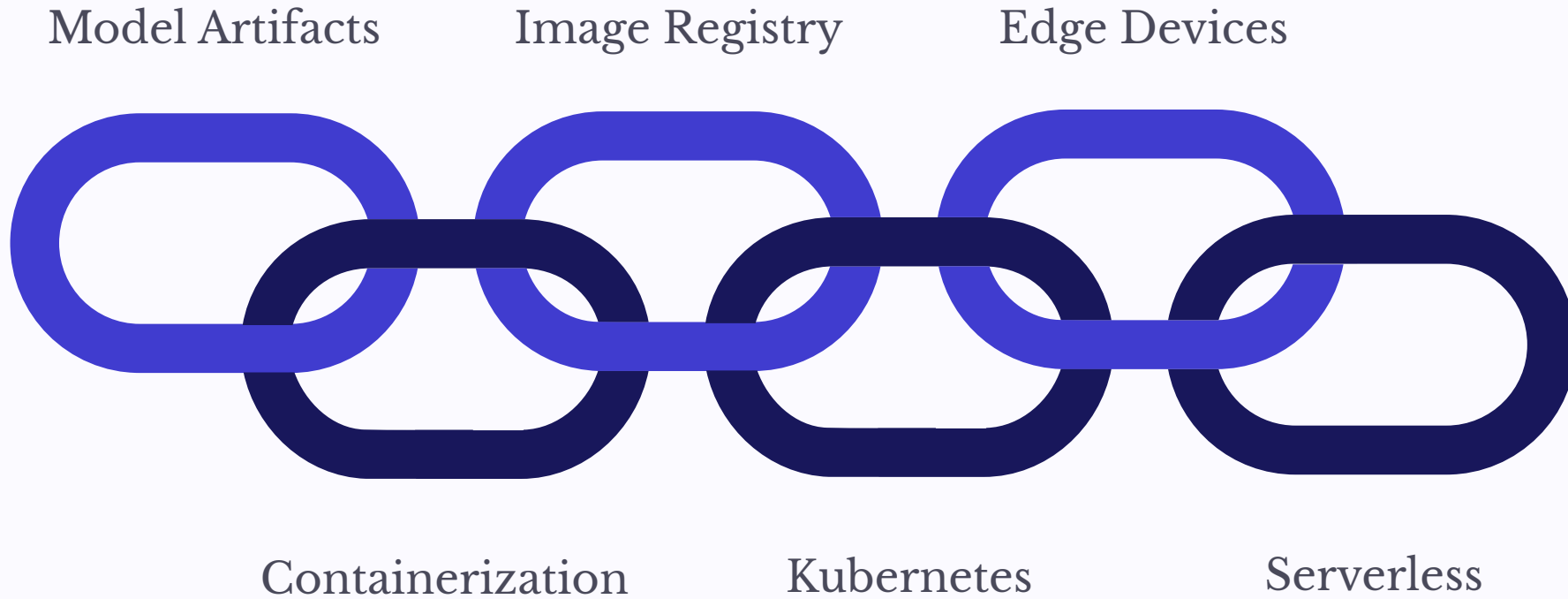


Power Draw

Energy efficient

This case proves container [portability from cloud to edge](#) without code changes

Container Deployment Workflow



Container = package once, deploy anywhere



Key Lessons

Portability

Containers provide consistent runtime across environments

- From laptop to production
- Cloud to edge continuity

Deployment Flexibility

Inference servers streamline serving

- Scaling and monitoring
- Framework-specific optimizations

Environment Control

Dockerfile defines exact runtime dependencies

- Library versions pinned
- System packages included

Versioning

Registries enable reproducible deployments

- Model history tracked
- Simple rollbacks when needed

Summary: Container Best Practices



Package Model & API Together

Combine inference code, dependencies, and model weights in a single container for simplified deployment



Version & Document

Tag containers with meaningful versions and document dependencies for traceability



Optimize for Performance

Use ONNX/TensorRT and multi-stage builds to reduce container size and improve inference speed



Test Across Environments

Verify container works identically in development, staging, and production environments

Containers form the [foundation for reliable MLOps](#) and are essential for your Capstone infrastructure

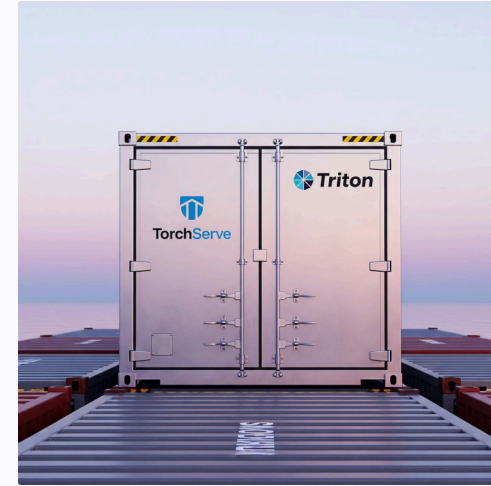
Discussion

If you were deploying your Capstone, would you use...



Lightweight FastAPI containers

- Simple implementation
- Full customization control
- Lower resource requirements
- Faster startup time



Heavy-duty inference server

- Production-grade features
- Better scaling capabilities
- Built-in monitoring
- Performance optimizations

What factors would influence your decision?